

Informe

Programacion I

The logo of the Universidad Tecnológica Nacional (UTN) is displayed. It consists of a stylized symbol on the left, followed by a vertical line, and then the letters 'UTN' in a bold, sans-serif font.

Estudiantes:

- Pablo Funes
- Valentino Rossi

Profesores:

- Cinthia Rigoni
- Ariel Enferrel

Fecha de entrega: XX/06/2026

Indice

Introduccion	-----	3
Base de datos:	-----	3
Variable	-----	3
Listas	-----	3
Diccionarios	-----	3
Estructuras condicionales:	-----	4
if	-----	4
elif	-----	4
else	-----	4
Estructuras repetitivas:	-----	5
while	-----	5
for	-----	5
Funciones	-----	6
Manejo de errores:	-----	6
try	-----	6
except	-----	6
finally	-----	7
Tipos de errores	-----	7
Manejo de archivos	-----	8
Otras funciones nativas de python:	-----	8
strip()	-----	8
split()	-----	9
.join()	-----	9
Librearias:	-----	9
csv	-----	9
os	-----	9
Questionary	-----	10
Decisiones Técnicas y Arquitectura	-----	11
Diagrama de flujos	-----	12
Ejemplos	-----	12
Conclusiones	-----	15
Bibliografía	-----	16
Link al repositorio del proyecto	-----	16
Link del vídeo explicativo	-----	16

Introducción:

Todo lo explicado en este informe, fue usado en la practica, esto incluye funciones internas de Python y externas. Se detalla cada uso con el fin de dar a entender al lector lo que puede reconstruir teniendo en cuenta la explicación del informe. Este informe busca que sea más una guía que una simple explicación.

Variables:

Son contenedores de datos, que se alojan en memoria con un “nombre” de referencia. Estos datos pueden ser strings, int, floats y bool.

```
variable = "Hola"
```

Variable con un dato string.

```
variable = 1.5
```

Variable con un dato float.

```
variable = 10
```

Variable con un dato int.

```
variable = True
```

Variable con un dato bool.

A las variables que posean datos del tipo int y floats. Se les puede hacer operaciones matemáticas.

Operador	Función
+	Suma
-	Resta
*	Multiplicación
/	División
//	División entera
%	Módulo (resto)
**	Potencia

Ejemplo N°1:

```
a = 10
b = 5

resta = a - b
print(resta) # salida: 5
```

Ejemplo N°2:

```
a = 5
b = 10

multiplicacion = a * b
print(multiplicacion) # salida 50
```

Estas mismas variables se pueden “llamar” dentro de un print(el “print” muestra por consola un mensaje), usando print(f”), a esto se le dice “fstring” y la variable se llama dentro de llaves {} :

```
division = a / b
print(f"El resultado es: {division}")
```

Listas:

Es una estructura de datos que permite almacenar múltiples elementos en una sola variable. Es mutable, ordenada(se puede acceder por índice) y puede contener datos de diferentes tipos (números, textos, booleanos u otras listas/diccionarios). Las listas se forman colocando los “datos” separados por comas entre corchetes. Es decir:

```
lista = ["Hola", "Adios", 80, 5.5]
```

```
variable1 = "Saludos"
variable2 = 80

lista = [variable1, variable2]
```

Diccionarios:

Es una estructura de datos que permite almacenar información en formato de pares clave-valor. A diferencia de las listas, donde los elementos se organizan por posiciones numéricas, en un diccionario accedes a los datos a través de una clave (que puede ser un texto, número o tupla, es decir cualquier inmutable) .Su formato es {clave:valor,otra_clave: otro_valor}. Es decir:

```
diccionario = {"hola": 4}
diccionario2 = {"adios": "saludo"}
```

Llave

Valor

Para recorrer un diccionario podemos usar `.keys()` para las llaves, `.values()` para los valores y `.items()` para los dos:

```
diccionario = {"hola": 4, "rayos": 10, "santos": 80}

claves = diccionario.keys()
valores = diccionario.values()
claves_valores = diccionario.items()

print(list(claves))
print(list(valores))
print(list(claves_valores))

#salida:
#['hola', 'rayos', 'santos']
#[4, 10, 80]
#[('hola', 4), ('rayos', 10), ('santos', 80)]
```

if, elif y else:

Python usa la palabra clave `if` para definir una condición, y opcionalmente se pueden agregar `else` y `elif` para manejar distintos casos. Por medio de ellas es posible que el programa defina qué bloques de código ejecutar primero o directamente no hacerlo.

Python usa la indentación para indicar que pertenece a un bloque `if`:

```
variable = 0
if variable == 0:
    print("La variable es igual a cero.")
elif variable == 1:
    print("La variable es igual a uno.")
else:
    print("La variable no es igual a cero ni a uno.")
```

Las indentaciones serían esos espacios que tiene el `print`. Es como si fuera una sangría.

Poseemos operadores que nos sirven para poner más condiciones de diferentes formas:

Operador	Definición	Ejemplo	Resultado
<code>==</code>	Igual a	<code>5 == 5</code>	True
<code>!=</code>	Distinto de	<code>5 != 3</code>	True
<code>></code>	Mayor a	<code>5 > 8</code>	False
<code><</code>	Menor a	<code>3 < 1</code>	False
<code>>=</code>	Mayor o igual a	<code>5 >= 4</code>	True
<code><=</code>	Menor o igual a	<code>4 <= 2</code>	False
<code>and</code>	Y, solo da True si todas son True.	<code>True and False</code>	False
<code>or</code>	O, solo da False si todas son False.	<code>False or True</code>	True
<code>not</code>	No, Invierte el valor.	<code>Not False</code>	True

While “variable bool”:

Es un ciclo el cual se repite siempre y cuando la variable devuelva “True”. En el caso de que la variable devuelva “False” el ciclo no se ejecuta.

```
while True:
    print("Hola")
```

Se da un ciclo infinito, debido a que no se cambia el valor del True.

```
while False:
    print("Hola")
```

No se ejecuta el código indentado al while debido a que el valor es False.

```
contador = 3
while contador != 0:
    print("Hola")
    contador -= 1
```

El código se ejecuta hasta que la variable “contador” no sea diferente a cero. No se da un bucle infinito debido a que con cada vuelta se le resta uno a “contador”.

```
while True:
    print("Adios")
    break
```

En un ciclo while True podemos usar “break” para detenerlo

Ciclo for:

El bucle for permite repetir una acción un número determinado de veces. A diferencia de while, acá sabemos de antemano cuántas veces se repetirá el código.

```
for variable in range (rango_inicial, rango_final):
    # salida 1
    # salida 2
    # salida 3
```

```
for i in range (0, 4):
    print(i)
# salida:
# 0
# 1
# 2
# 3
```

Funciones:

Una funcion es un bloque de código reutilizable que realiza una tarea específica. En lugar de escribir el mismo código varias veces, podemos definir una función y llamarla cuando sea necesario.

Esto sirve bastante para tener un código más ordenado.

```
def saludo():  
    print("Hola.")
```

De esta forma, nosotros definimos una función. En este caso la función "saludo"

```
saludo()
```

Para llamar a una función, escribimos su nombre y adelante le ponemos parentesis.

Las funciones pueden trabajar con datos ya existentes. Solo hace falta llamar ese parametro para la función:

```
def suma(a, b):  
    resultado = a + b  
    return resultado
```

```
a = 5  
b = 10  
  
valor = suma(a, b)  
print(valor)  
  
# salida:  
# 15
```

En este caso, a la función suma le dijimos que use los parámetros "a" y "b" para que realice una simple suma. A la hora de usar la función, debemos de llamar esos parámetros. Colocando el nombre de las variables. En el caso de que no las llamemos, nos dará un error por falta de error.

Los parámetros recibidos no tienen que llevar si o si el mismo nombre que las variables. También podemos darle un valor sin necesidad de una variable.

```
def saludar(nombre):  
    print(f"Hola {nombre}")
```

```
saludar("Juanita")  
#salida:  
# Hola Juanita
```

try except:

El bloque try contiene el código que puede lanzar una excepción. Una excepción es un tipo especial de error que ocurre mientras el programa se está ejecutando.

```
try:  
    # Código que podría fallar:  
    suma = 5 + 2  
except error_posible:  
    # Parte del código que se ejecuta al saltar un error.  
    print("No es posible realizar esa suma.")
```

Se pueden usar múltiples except y también except no especificados:

```
try:  
    suma = "5" + 2  
except TypeError:  
    print("No es posible realizar una suma con string e int.")  
except:  
    print("Ocurrió un error no especificado")
```

Tipos de errores:

◆ZeroDivisionError

Se produce cuando intentamos dividir un número por cero.

```
try:
    division = 8 / 0
except ZeroDivisionError:
    print("No es posible dividir por cero.")
```

◆ValueError

Ocurre cuando intentamos convertir datos a un tipo incompatible. Por ejemplo, convertir letras a número.

```
try:
    int(input("Ingrese su letra inicial por favor: "))
except ValueError:
    print("Las letras no se pueden convertir a numeros.")
```

◆FileNotFoundError

Aparece cuando se intenta abrir un archivo que no existe.

```
try:
    archivo = open("archivo.txt", "r")
except FileNotFoundError:
    print("No se encontro un archivo con ese nombre.")
```

◆KeyError

Se lanza cuando tratamos de acceder a una clave que no existe en un diccionario.

```
granja = {"materia prima": "trigo", "producto": "harina"}
try:
    print(granja["precio"])
except KeyError:
    print("No se encontro una clave con ese nombre.")
```

◆IndexError

Se genera al intentar acceder a una posición que no existe en una lista o tupla.

```
lista = ["Hola", "Bola", "Sola", "Ola"]
try:
    print(lista[4])
except IndexError:
    print("No hay ningun elemento en el index 4")
```

◆TypeError

Se lanza cuando se realizan operaciones entre tipos de datos incompatibles.

```
try:
    multiplicacion = "10" * 5
except TypeError:
    print("No se puede realizar calculos matematicos entre strings e ints.")
```

◆PermissionError

Se genera cuando no hay permisos suficientes para abrir el archivo.

```
try:
    archivo = open("archivo.txt", "r")
except PermissionError:
    print("No se puede acceder al archivo debido a falta de permisos.")
```

Además de los bloques try y except, existe un bloque especial llamado finally. Este bloque se ejecuta siempre, sin importar si ocurrió una excepción o no.

```
try:
    suma = 10 + 67
except TypeError:
    print("No se pueden hacer operaciones matematicas con valores de diferente tipo.")
finally:
    print("Intento de tarea realizado.")
```

Manejo de archivos:

Modos de apertura:

"r" Abre un archivo para lectura en modo texto. El archivo debe existir, si no, lanza un error.	"rb" Abre un archivo para lectura en modo binario.	"r+" Abre un archivo para lectura y escritura en modo texto. El archivo debe existir. No borra su contenido.
"w" Abre un archivo para escritura en modo texto. Si el archivo ya existe, borra su contenido. Si no existe, lo crea.	"wb" Abre un archivo para escritura en modo binario. Si el archivo ya existe, se borra. Si no existe, se crea.	"w+" Abre un archivo para lectura y escritura, en modo texto. Sobrescribe el contenido del archivo si ya existe.
	"a" Abre un archivo para agregar contenido al final. Si el archivo no existe, lo crea. No borra el contenido previo.	

Para su uso, basta con nombrar al archivo como una variable, escribir `open(nombre_archivo, modo)`:

```
archivo = open("archivo.txt", "r")
```

Los archivos que podemos abrir son varios, como por ejemplo: .txt .csv .json y otros más.

En el modo van los ya mencionado "modos de apertura": "r", "w", "a", entre otros.

Para cerrar un archivo, usamos `.close()`

```
archivo.close()
```

Al cerrar un archivo podemos evitar perdida de datos, bloqueos de sistema, etc.

Para trabajar de una forma más segura, usamos `with`. Abre el archivo y garantiza que se cerrará automáticamente al finalizar el bloque, incluso si ocurre un error dentro del mismo.

```
with open("archivo.txt", "r") as archivo:
    for line in archivo:
        print(f"{line}\n")
```

Metodos para "w"	Metodos para "r"
write() : permite escribir una sola linea.	read() : Lee todo el contenido del archivo como una unica cadena de texto
No existe un <code>writeline()</code>	readline() : Lee una sola linea del archivo.
writelines() : permite escribir varias lineas	readlines() : Lee todo el contenido del archivo como una lista de cadenas.

Otras funciones nativas de python:

◆ `strip()`:

La función `strip()` se usa para eliminar los caracteres en blanco al inicio y al final de una cadena de texto. Tambien se le pueden pasar caracteres especificos para eliminar.

```
texto = "  En esta oracion estoy poniendo espacios al inicio y al final  "
texto_sin_espacios = texto.strip()
print(texto_sin_espacios)

# salida:
# En esta oracion estoy poniendo espacios al inicio y al final
```

◆split():

La función split() divide una cadena en una lista de partes, usando como criterio de separación un carácter específico.

```
texto = "Hola,Adios,10,Ocho"
partes = texto.split(",")
print(partes)

# salida:
# ['Hola', 'Adios', '10', 'Ocho']
```

◆.join():

.join() es un método que pertenece a los strings y se usa para unir elementos de una secuencia (como listas o tuplas) en un solo string, usando el string que llama a .join() como separador:

```
palabras = ["Hola", "adios"]
resultado = " ".join(palabras)
print(resultado)

# salida:
# Hola adios
```

Librerías:

Las librerías son modificaciones que agregan código, con la finalidad de realizar acciones que sin ellas no podríamos dentro de python.

Para importar una librería, usamos import:

```
import libreria
```

Python posee librerías, estas librerías son clasificadas como internas, también hay librerías externas: Las que no pertenecen a python y requieren de una instalación.

Usando pip instalamos "librería" dentro de la consola. Podemos instalar librerías:

```
PS C:\Users\Usuario> pip install colorama
```

Las librerías puestas en práctica son:

◆Librerías internas:

Python posee librerías, en el proyecto hemos aprovechado algunas de ellas:

◆csv:

El módulo csv es una herramienta integrada para leer y escribir archivos de texto plano estructurados en columnas. Evita los errores comunes de hacerlo manualmente (como manejar comas dentro del texto o saltos de línea) procesando estos formatos de manera automática y segura.

Ofrece dos enfoques principales:

- Basado en listas (reader / writer): Ideal para procesar datos de forma secuencial y eficiente, manejando cada fila como una lista de elementos.
- Basado en diccionarios (DictReader / DictWriter): Perfecto si el archivo tiene encabezados, permitiendo acceder a los datos mediante el nombre de la columna.

Como si se tratara de un archivo txt, lo abrimos de la misma forma, con algunas excepciones en el código:

```
import csv

with open("archivo.csv", "r", encoding="utf-8", newline="") as archivo:
    lector = csv.reader(archivo)
```

Encoding="utf-8": nos sirve para detectar caracteres que python no detecta normalmente. Ej: Ñ, ñ, á, é, ö.

Newline="": nos sirve para que no se tomen las líneas vacías.

◆ os:

os tiene muchas funciones, nosotros la usamos solamente para limpiar la consola.
La indicacion para limpiar la consola es la siguiente:

```
import os

os.system('cls' if os.name == 'nt' else 'clear')
```

◆ Librerias externas:

◆ Questionary:

Questionary es una libreria que permite el uso de las flechas del teclado para manejarse entre las opciones de un menu. El enter permite seleccionar esa opcion.

Con `questionary.select().ask()` podemos crear un menu, y como devuelve un valor, se lo podemos asignar a una variable:

```
import questionary

opcion = questionary.select("¡Hola!", ["1-Hola;", "2-Adios;", "3-Callate."]).ask()

print(opcion)
```

Lo primero que va como string es la pregunta que queremos hacer, despues, entre corchetes escribimos las opciones separadas por comas. Estas opciones pueden ser strings, ints, floats, etc.

Dentro de la consola se ve así:

```
? ¡Hola! (Use arrow keys)
» 1-Hola;
  2-Adios;
  3-Callate.
```

Si nosotros apretamos una sola vez la flecha de abajo, empezamos a navegar por las opciones disponibles:

```
? ¡Hola! (Use arrow keys)
  1-Hola;
» 2-Adios;
  3-Callate.
```

Si apretamos enter, entonces seleccionaremos la opcion:

```
? ¡Hola! 2-Adios;
2-Adios;
PS C:\Users\Usuario>
```

El `print(opcion)` imprimio la opcion seleccionada. Arriba, al lado de la pregunta podemos ver la opcion seleccionada también.

Decisiones Técnicas y Arquitectura

Arquitectura General del Sistema

Para el desarrollo de este sistema de gestión de países se optó por una arquitectura modular, donde cada funcionalidad principal fue implementada mediante funciones independientes. Esta decisión permitió organizar mejor el código, facilitar su mantenimiento y mejorar la legibilidad del programa.

La aplicación trabaja principalmente sobre una lista de diccionarios, donde cada diccionario representa un país y almacena los siguientes datos:

- Nombre del país.
- Población.
- Superficie.
- Continente al que pertenece.

Además, se decidió utilizar un archivo CSV como mecanismo de persistencia de datos. De esta manera, la información cargada por el usuario puede conservarse incluso después de cerrar el programa. Al iniciar la ejecución, los datos son cargados desde el archivo y, al finalizar, se vuelven a guardar automáticamente.

Otro aspecto importante de la arquitectura fue la implementación de excepciones personalizadas. Estas excepciones permiten detectar y manejar errores específicos del sistema, como intentos de cargar países duplicados, listas vacías, datos inexistentes o valores numéricos inválidos. Gracias a este enfoque, el programa resulta más robusto y ofrece mensajes de error claros para el usuario.

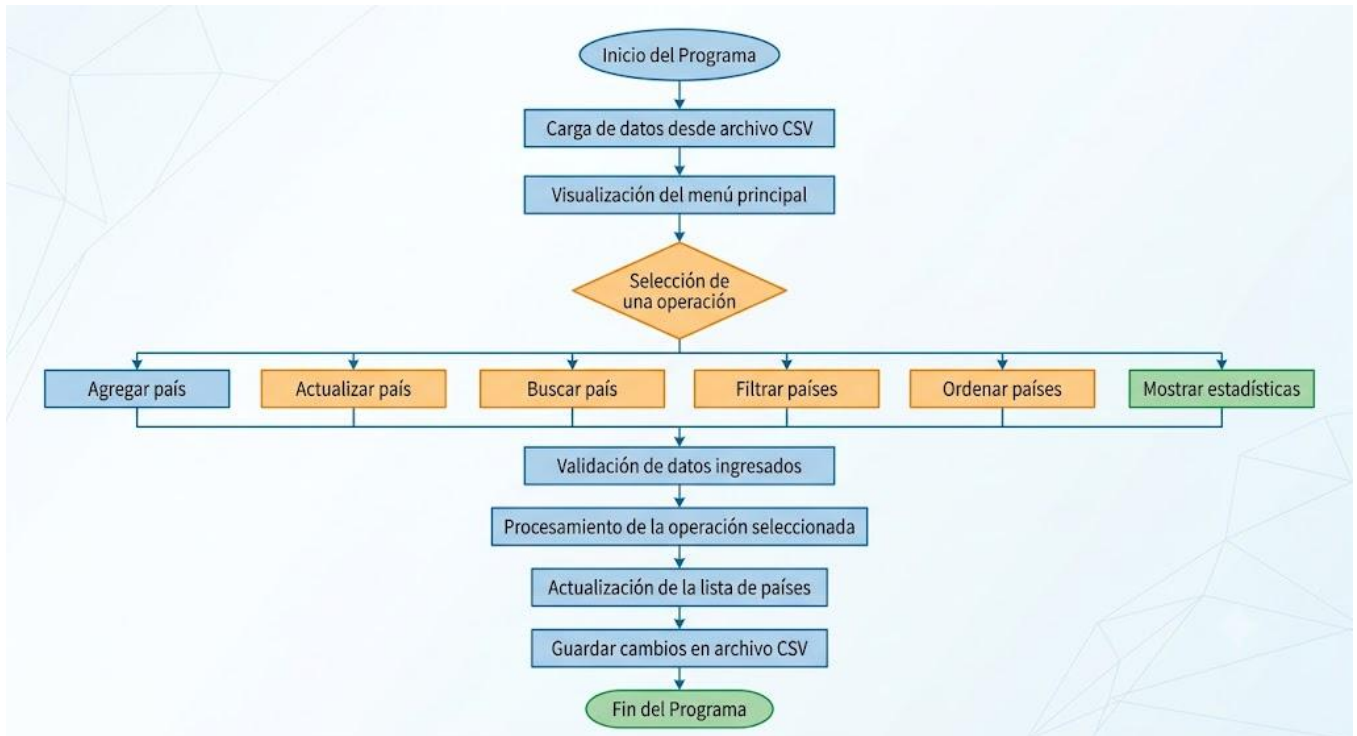
También se incorporó la biblioteca Questionary para la creación de menús interactivos. Esta herramienta mejora considerablemente la experiencia de uso al permitir seleccionar opciones mediante un menú visual en lugar de ingresar números manualmente.

Ej. Menu con Questionary:

```
? Seleccione la acción a realizar: (Use arrow keys)
» 1. Agregar un país.
   2. Actualizar los datos de Población y Superficie de un País.
   3. Buscar un país por nombre.
   4. Filtrar países.
   5. Ordenar países.
   6. Mostrar estadísticas.
   7. Salir.
```

Diagrama de Flujo de las Operaciones Principales

El funcionamiento general del sistema puede resumirse mediante el siguiente flujo lógico:



Cada una de estas operaciones incluye mecanismos de validación para evitar errores de entrada y garantizar la integridad de la información almacenada.

Capturas de Pantalla de Ejecución

A continuación se presentan ejemplos de las principales funcionalidades ejecutadas durante las pruebas del sistema.

Ejemplo 1: Agregar un país

```
? Seleccione la acción a realizar: 1. Agregar un país.  
Ingrese el nombre del país: Colombia  
Ingrese la cantidad de población: 33344525235  
Ingrese la superficie del país: 442342343  
Ingrese el continente al que pertenece el país: America  
¡Se agregó correctamente el país!  
Se guardaron correctamente los datos.  
Presione ↵Enter para continuar
```

En esta operación se verifica que el nombre no exista previamente en la lista y que los valores numéricos sean mayores que cero.

Ejemplo 2: Actualizar los datos de Población y Superficie de un País.

```
? Seleccione la acción a realizar: 2. Actualizar los datos de Población y Superficie de un País.
¿Cual país desea actualizar?: Argentina
¿Cual sera la nueva poblacion para Argentina?: 3334442222
¿Cual sera la nueva superficie para Argentina?: 44442278865
País actualizado correctamente.
Se guardaron correctamente los datos.
Presione ↵Enter para continuar
```

Ejemplo 3: Búsqueda de países

```
? Seleccione la acción a realizar: 3. Buscar un país por nombre.
Ingrese el nombre del país que desea buscar: Argen
Nombre del país: Argentina | Población: 45538401 | Superficie: 2780400 | Continente: América del Sur
Presione ↵Enter para continuar
```

El sistema permite realizar búsquedas parciales o exactas.

La búsqueda implementada ignora diferencias entre mayúsculas, minúsculas y caracteres acentuados.

Ejemplo 4: Filtrar Países

```
? Seleccione la acción a realizar: 4. Filtrar países.
? Eliga el filtro: (Use arrow keys)
» 1) Continente;
   2) Rango de población;
   3) Rango de superficie;
   4) Volver atras.
```

```
? Seleccione la acción a realizar: 4. Filtrar países.
? Eliga el filtro: 1) Continente;
? Seleccione el continente: África
-Angola.
-Argetia.
-Benín.
-Botsuana.
-Burkina Faso.
```

Ejemplo 5: Ordenamiento

El usuario puede ordenar los países según distintos criterios.

```
? Seleccione la acción a realizar: 5. Ordenar países.  
? Ordenar países por: (Use arrow keys)  
» 1. Nombre  
   2. Población  
   3. Superficie  
   4. Volver al menú principal
```

Ordenar de forma:

```
? Seleccione la acción a realizar: 5. Ordenar países.  
? Ordenar países por: 1. Nombre  
? Ordenar de forma: (Use arrow keys)  
» 1. Ascendente  
   2. Descendente  
   3. Volver al menú principal
```

Se utilizó el método `sort()` de Python junto con funciones lambda para realizar el ordenamiento de forma eficiente.

Ejemplo 6: Estadísticas

El sistema calcula automáticamente información estadística a partir de los datos cargados.

```
? Seleccione la acción a realizar: 6. Mostrar estadísticas.  
  
El país con mayor población es: China, con una cantidad de: 1422584933 habitantes.  
El país con menor población es: Tuvalu, con una cantidad de: 12000 habitantes.  
El promedio de población es: 43939744.89204545.  
El promedio de superficie es: 729825.6590909091.  
Cantidad de países en..  
  América: 0;  
  Europa: 41;  
  Asia: 42;  
  África: 47;  
  Oceanía: 12.  
Presione ↵Enter para continuar
```

Estas estadísticas permiten obtener una visión general de los datos almacenados y constituyen una de las funcionalidades más importantes del proyecto.

Conclusión Técnica

Durante el desarrollo del sistema se aplicaron conceptos fundamentales de programación vistos durante el primer año de la carrera, tales como estructuras de datos, manejo de archivos, validación de entradas, excepciones personalizadas, funciones, listas, diccionarios, ordenamiento y filtrado de información.

La utilización de una estructura modular permitió separar responsabilidades y simplificar las tareas de prueba y mantenimiento. Asimismo, el uso de archivos CSV proporcionó una solución sencilla y eficiente para el almacenamiento persistente de datos. Como resultado, se obtuvo una aplicación funcional capaz de gestionar información de países de manera segura, organizada y fácil de utilizar.

Obstáculos Técnicos y Cómo los Resolvimos

A lo largo del proyecto nos chocamos con varias piedras en el camino, sobre todo en lo que respecta a la robustez del código y a lograr que la interfaz por consola fuera limpia. El primer gran desafío fue la persistencia y manipulación del archivo CSV. Hacer que la lectura y escritura con el modulo CSV funcionara sin problemas implicó ponernos estrictos con los bloques try-except. Tuvimos que prever desde que el archivo no existiera, hasta que vinieran datos rotos o vacíos en las columnas, decidiendo ignorar esas líneas conflictivas para evitar que todo el programa colapsara.

Otro obstáculo fue la validación y limpieza de los inputs. Al principio, si un usuario metía un espacio de más, un número donde iba una letra o mezclaba mayúsculas con tildes, las búsquedas y los filtros fallaban. Para solucionar esto de raíz, nos pusimos a armar funciones específicas de validación de inputs y una función clave para quitar tildes y normalizar textos. Con esto logramos estandarizar las comparaciones (evitando que el sistema tomara "América" y "america" como cosas distintas) y le dimos una experiencia de usuario mucho más prolija e interactiva usando la librería `questionary`.

Por último, el orden del código nos costó su tiempo. Separar las funciones en un módulo y dejar el menú principal en otro nos obligó a entender bien cómo viajan los datos y cómo pasan las listas de una función a otra. La creación de nuestras propias excepciones fue la solución ideal para manejar las cancelaciones de carga a mitad de camino.

Conclusión y Reflexiones Finales

El desarrollo de este Trabajo Final Integrador para la materia Programación I ha representado, sin lugar a dudas, el desafío más importante de nuestro primer semestre en la facultad. Esta instancia significó la oportunidad real de ponernos a prueba y de ver si éramos capaces de tomar toda la teoría acumulada en las clases y llevarla a código real, funcional y estructurado. También significó una oportunidad para trabajar en grupo y de forma remota con un compañero de equipo, asemejándose a la realidad del trabajo, ya que en el ámbito laboral y real no se trabaja como individuo si no como grupo. Pasamos de resolver pequeños algoritmos aislados en las guías prácticas a diseñar un sistema integral en Python para la gestión, manejo y persistencia de datos demográficos de países nos cambió por completo la perspectiva de la programación que teníamos. Nos obligó a madurar el pensamiento lógico y a entender que un buen programa no es aquel que solo cumple con el camino feliz cuando todo sale bien, sino el que está preparado para resistir los errores del usuario, las fallas del entorno y las inconsistencias de los datos sin colapsar en el intento.

Bibliografía:

Funciones y librerías nativas:

Estructuras secuenciales (variables, operadores.):

[Estructuras secuenciales part-1.pdf](#)

[Estructuras secuenciales part-2.pdf](#)

Lista:

[3. Modelo de datos — documentación de Python - 3.14.5](#)

Diccionarios:

[Diccionarios - TUPaD.docx](#)

Estructuras condicionales:

[Condicionales](#)

[Operadores Logicos](#)

[Operadores Relacionales](#)

Estructuras repetitivas:

[Repetitivas-part-1.pdf](#)

[Repetitivas-part-2.pdf](#)

Manejo de errores:

[Tipo de Errores.docx](#)

[Try Except.docx](#)

Manejo de archivos:

[1 - Introducción Archivos .pdf](#)

[Apertura, lectura, escritura, cierre de archivos.pdf](#)

[Apunte - archivos.pdf](#)

Otras funciones nativas:

[strip\(\),split\(\),join\(\).pdf](#)

Librería questionnaire:

[questionnaire · PyPI](#)

Datos de países:

[Microsoft Copilot](#)

Con informaciones adicionales añadidas en base a lo aprendido en clases de programación I

Link al repositorio del proyecto

<https://github.com/heroeseb/UTN-TUP-TPI-Programacion-1-Funes-Rossi.git>

Link del video

<https://youtu.be/fZ1HchLIDAY>